

## **Příprava – AJAX, REST, JSON, XML, chybové a návratové kódy, vracení dat, Web API, význam metod**

### **1. Základní pojmy**

#### **Web API**

- Rozhraní, přes které spolu komunikují aplikace (klient ↔ server)
- Typicky přes internet pomocí HTTP
- Používá se např. v desktopových aplikacích pro načítání dat ze serveru

#### **REST (RESTful API)**

- Architektonický styl pro tvorbu API
  - Využívá HTTP (GET, POST...)
  - Pracuje se „zdroji“ (např. /users/1)
  - Vlastnosti:
    - **Stateless** – server si nepamatuje stav (každý request je samostatný)
    - **Jednotné rozhraní** – stejné principy pro všechna API
  - Další architektury, například GraphQL
- 

### **2. HTTP metody (význam)**

- **GET**
  - Získání dat
  - Nemění data na serveru
- **POST**
  - Vytvoření nového záznamu
  - Posílají se data (např. JSON)
- **PUT**
  - Přepsání celého objektu
- **PATCH**
  - Částečná úprava objektu
- **DELETE**

- Smazání dat
  - **HEAD**
    - Stejně jako GET, ale bez těla (jen hlavičky)
  - **OPTIONS**
    - Zjištění, co API podporuje (důležité pro CORS)
- 

### 3. AJAX

- **Asynchronous JavaScript and XML**
- Technika pro komunikaci se serverem **bez reloadu aplikace**
- Dnes se používá hlavně s JSON (XML považováno za deprecated)

Použití:

- načítání dat z API na pozadí
  - desktop app (např. C# HttpClient, Electron, webview)
- 

### 4. JSÓN vs XML

#### JSON

- lehký, rychlý, přehledný
- používá se nejčastěji v REST API

```
{ "name": "Jan", "age": 20 }
```

#### XML

- složitější, více „ukecaný“
- podporuje atributy a schémata
- dnes se s ním setkáme postupně méně a méně

```
<user>
  <name>Jan</name>
</user>
```

---

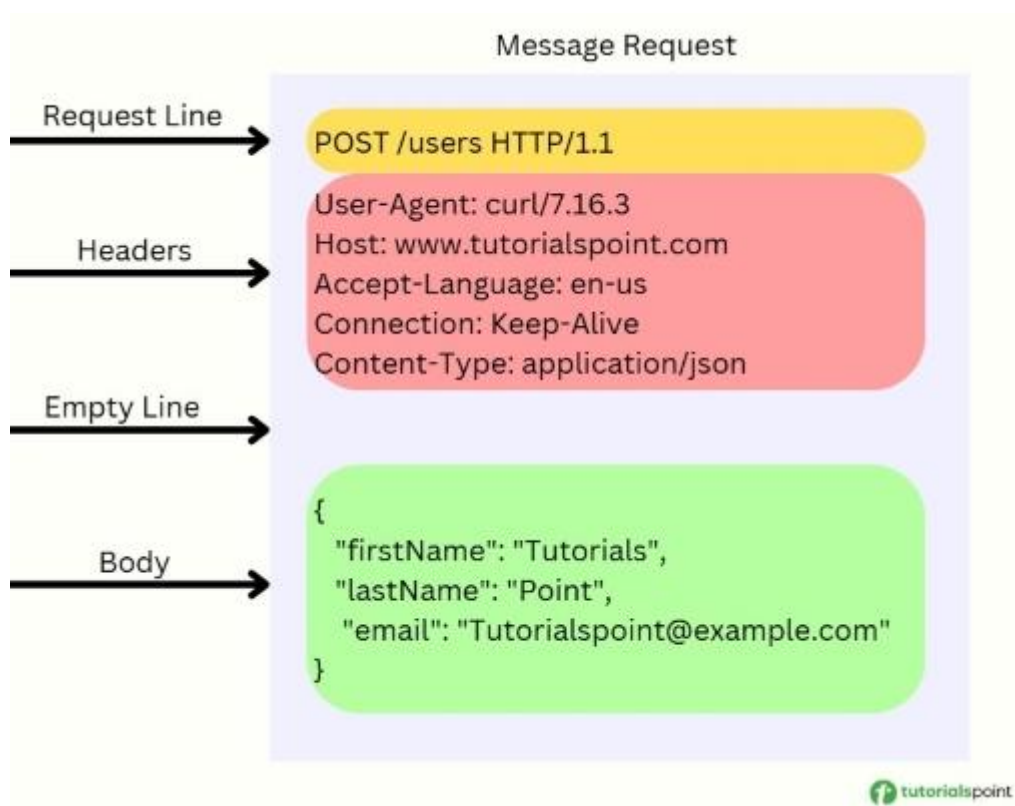
### 5. HTTP request a response

**Request (požadavek):**

- metoda (GET, POST...)
- URL (např. /api/users)
- hlavičky (Content-Type, Authorization)
- tělo (např. JSON)

### Response (odpověď):

- status kód (200, 404...)
- hlavičky
- data (většinou JSON)



## 6. Stavové (HTTP) kódy

Rozdělení:

- **1xx** – informativní
- **2xx** – úspěch
- **3xx** – přesměrování
- **4xx** – chyba klienta
- **5xx** – chyba serveru

Přehled:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

---

## 8. Typické chyby

- 400 → špatný JSON / parametry
- 401 → není přihlášen
- 403 → nemá práva
- 404 → neexistuje
- 500 → chyba serveru

Klient by měl:

- ošetřit chyby (try/catch)
  - zobrazit zprávu uživateli
- 

## 9. Bezpečnost (základ)

- používat **HTTPS**
  - token v hlavičce (Authorization)
  - ne posílat hesla v URL
- 

## 10. Shrnutí (na potítko)

- REST = komunikace přes HTTP + práce se zdroji
- API = rozhraní mezi aplikacemi
- AJAX = asynchronní volání API
- JSON = hlavní formát dat
- HTTP metody = CRUD operace
- status kódy = informace o výsledku